AI Assisted Coding

Name: K. Sai Vardhan Reddy

Roll no:2503A51266

Assignment no:16

Task 1 – Employee Management Database

Task : Create relational tables with keys/constraints and write the required JOIN, aggregate, and analysis queries.

SQL Code

Task 1 - Employee Management Database

SQL > main.sql

```sql
-- MySQL | Task 1: Employee Management
CREATE DATABASE EmployeeDB;
USE EmployeeDB;

CREATE TABLE Departments (
  department_id INT PRIMARY KEY AUTO_INCREMENT,
  department_name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE Employees (
  employee_id INT PRIMARY KEY AUTO_INCREMENT,
  employee_name VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  department_id INT NOT NULL,
  FOREIGN KEY (department_id)
    REFERENCES Departments(department_id)
);

CREATE TABLE Salaries (
  salary_id INT PRIMARY KEY AUTO_INCREMENT,
  employee_id INT NOT NULL,
  salary DECIMAL(10,2) NOT NULL CHECK (salary >= 0),
  effective_date DATE NOT NULL,
  FOREIGN KEY (employee_id) REFERENCES Employees(employee_id)
);

INSERT INTO Departments(department_name)
VALUES ('IT'), ('HR'), ('Sales');
INSERT INTO Employees(employee_name,email,department_id)
VALUES ('Asha','asha@demo.com',1), ('Ravi','ravi@demo.com',1),
       ('Mina','mina@demo.com',2), ('Kiran','kiran@demo.com',3);
INSERT INTO Salaries(employee_id,salary,effective_date)
VALUES (1,70000,'2026-01-01'), (2,85000,'2026-01-01'),
       (3,50000,'2026-01-01'), (4,60000,'2026-01-01');

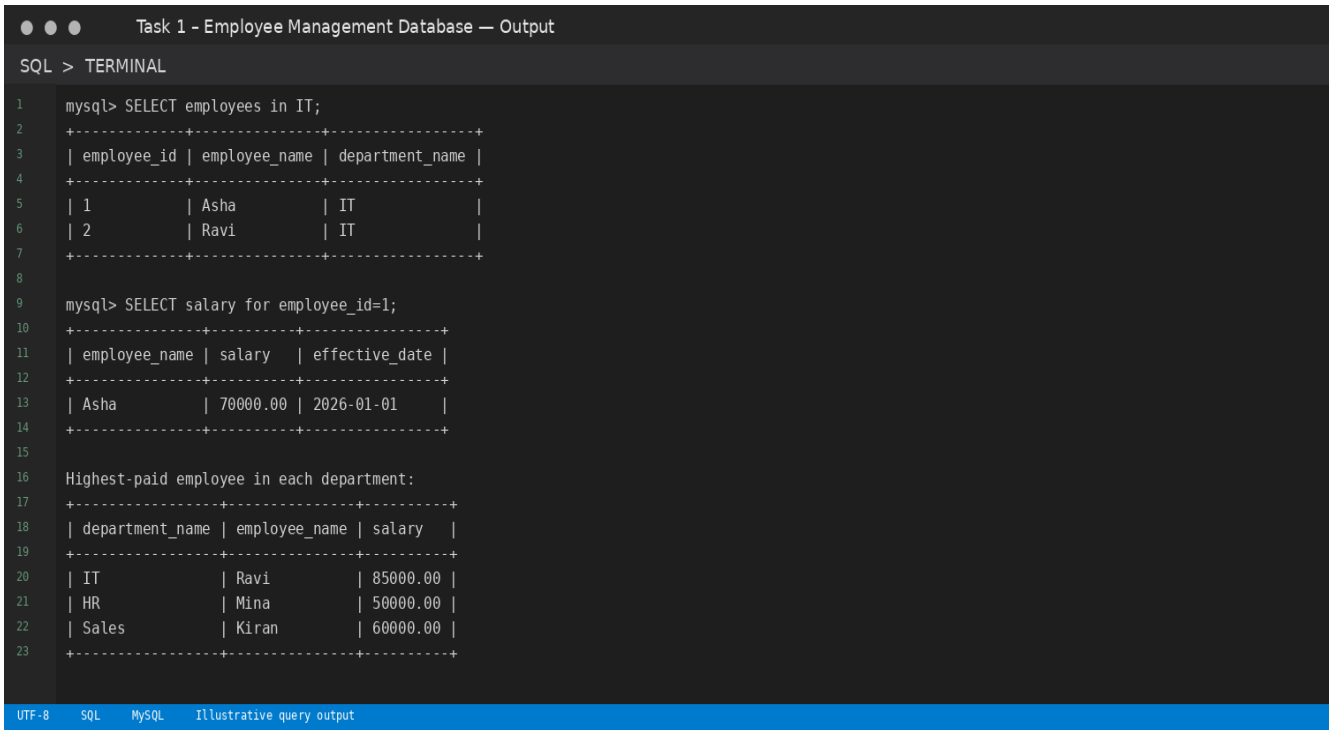
-- Employees in a department
SELECT e.employee_id,e.employee_name,d.department_name
FROM Employees e JOIN Departments d
ON e.department_id=d.department_id
WHERE d.department_name='IT';

-- Salary details for one employee
SELECT e.employee_name,s.salary,s.effective_date
FROM Employees e JOIN Salaries s ON e.employee_id=s.employee_id
WHERE e.employee_id=1;

-- Highest-paid employee in each department
SELECT department_name,employee_name,salary
FROM (
 SELECT d.department_name,e.employee_name,s.salary,
        ROW_NUMBER() OVER(PARTITION BY d.department_id
                          ORDER BY s.salary DESC) AS rn
 FROM Departments d JOIN Employees e
 ON d.department_id=e.department_id
 JOIN Salaries s ON e.employee_id=s.employee_id
) ranked
WHERE rn=1;
```

UTF-8 SQL MySQL Code example

Expected Output



Note: For the airline task, available seats are calculated from booked tickets. For the hotel task, room availability uses overlapping date ranges, and occupancy is calculated for the selected date.

Task 2 – Online Course Platform Database

Task : Create relational tables with keys/constraints and write the required JOIN, aggregate, and analysis queries.

SQL Code

Task 2 – Online Course Platform Database

SQL > main.sql

```sql
-- MySQL | Task 2: Online Course Platform
CREATE DATABASE CourseDB;
USE CourseDB;

CREATE TABLE Users (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE
);
CREATE TABLE Courses (
  course_id INT PRIMARY KEY AUTO_INCREMENT,
  course_title VARCHAR(150) NOT NULL,
  instructor VARCHAR(100) NOT NULL
);
CREATE TABLE Lessons (
  lesson_id INT PRIMARY KEY AUTO_INCREMENT,
  course_id INT NOT NULL,
  lesson_title VARCHAR(150) NOT NULL,
  lesson_order INT NOT NULL,
  FOREIGN KEY (course_id) REFERENCES Courses(course_id),
  UNIQUE(course_id, lesson_order)
);
CREATE TABLE Enrollments (
  user_id INT NOT NULL,
  course_id INT NOT NULL,
  enrolled_on DATE NOT NULL,
  PRIMARY KEY(user_id, course_id),
  FOREIGN KEY(user_id) REFERENCES Users(user_id),
  FOREIGN KEY(course_id) REFERENCES Courses(course_id)
);

INSERT INTO Users(full_name,email) VALUES
('Anu','anu@demo.com'),('Dev','dev@demo.com'),('Sam','sam@demo.com');
INSERT INTO Courses(course_title,instructor) VALUES
('Python Basics','Mr Rao'),('SQL Fundamentals','Ms Devi');
INSERT INTO Lessons(course_id,lesson_title,lesson_order) VALUES
(1,'Variables',1),(1,'Loops',2),(2,'SELECT Queries',1);
INSERT INTO Enrollments VALUES
(1,1,'2026-10-01'),(1,2,'2026-10-02'),(2,1,'2026-10-03');

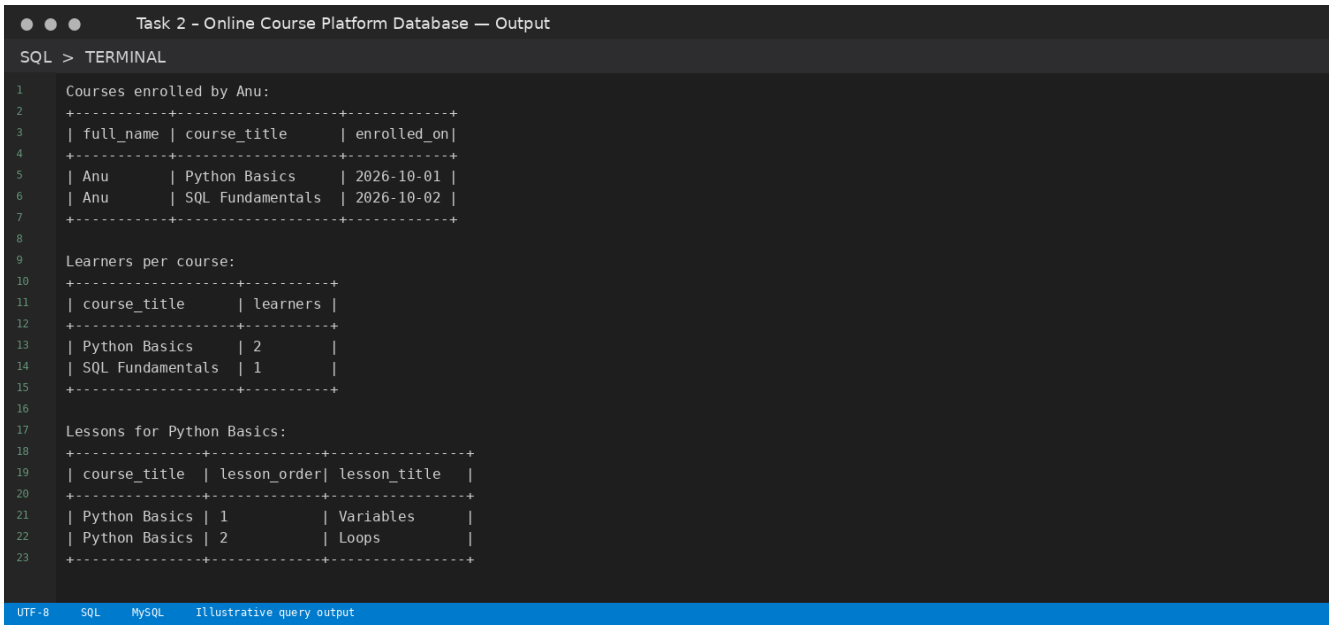
-- Courses enrolled by a user
SELECT u.full_name,c.course_title,e.enrolled_on
FROM Enrollments e JOIN Users u ON e.user_id=u.user_id
JOIN Courses c ON e.course_id=c.course_id
WHERE u.user_id=1;

-- Number of learners per course
SELECT c.course_title,COUNT(e.user_id) AS learners
FROM Courses c LEFT JOIN Enrollments e ON c.course_id=e.course_id
GROUP BY c.course_id,c.course_title;

-- Lessons for a course
SELECT c.course_title,l.lesson_order,l.lesson_title
FROM Courses c JOIN Lessons l ON c.course_id=l.course_id
WHERE c.course_id=1 ORDER BY l.lesson_order;
```

UTF-8 SQL MySQL Code example

Expected Output



Note: For the airline task, available seats are calculated from booked tickets. For the hotel task, room availability uses overlapping date ranges, and occupancy is calculated for the selected date.

Task 3 – Banking Transaction Database

Task : Create relational tables with keys/constraints and write the required JOIN, aggregate, and analysis queries.

SQL Code

Task 3 – Banking Transaction Database

SQL > main.sql

```sql
-- MySQL | Task 3: Banking Transactions
CREATE DATABASE BankingDB;
USE BankingDB;

CREATE TABLE Customers (
  customer_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_name VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE
);
CREATE TABLE Accounts (
  account_id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT NOT NULL,
  balance DECIMAL(12,2) NOT NULL DEFAULT 0 CHECK(balance >= 0),
  FOREIGN KEY(customer_id) REFERENCES Customers(customer_id)
);
CREATE TABLE Transactions (
  transaction_id INT PRIMARY KEY AUTO_INCREMENT,
  account_id INT NOT NULL,
  transaction_type ENUM('DEPOSIT','WITHDRAWAL') NOT NULL,
  amount DECIMAL(12,2) NOT NULL CHECK(amount > 0),
  transaction_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(account_id) REFERENCES Accounts(account_id)
);

INSERT INTO Customers(customer_name,email) VALUES
('Riya','riya@demo.com'),('Arun','arun@demo.com');
INSERT INTO Accounts(customer_id,balance) VALUES (1,12000),(2,800);
INSERT INTO Transactions(account_id,transaction_type,amount,transaction_date)
VALUES (1,'DEPOSIT',5000,'2026-10-20 10:00:00'),
       (1,'WITHDRAWAL',1500,'2026-10-21 11:00:00'),
       (2,'DEPOSIT',300,'2026-10-22 12:00:00');

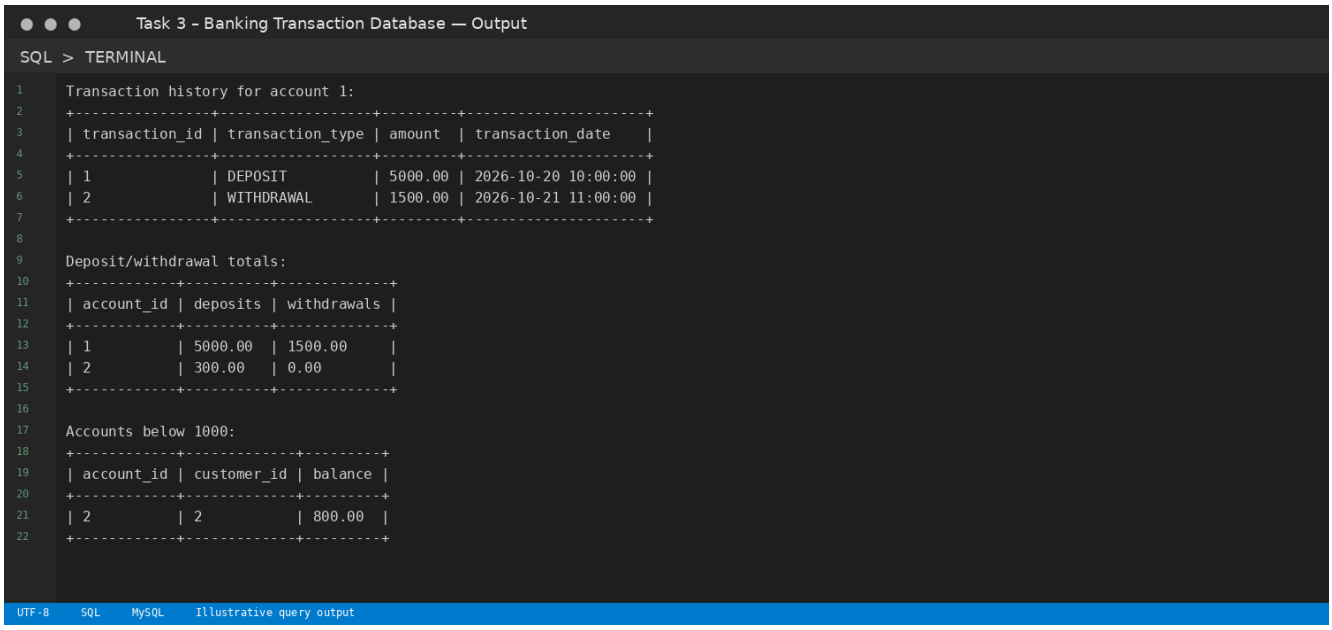
-- Transaction history for an account
SELECT transaction_id,transaction_type,amount,transaction_date
FROM Transactions WHERE account_id=1 ORDER BY transaction_date;

-- Total deposits and withdrawals
SELECT account_id,
 SUM(CASE WHEN transaction_type='DEPOSIT' THEN amount ELSE 0 END) AS deposits,
 SUM(CASE WHEN transaction_type='WITHDRAWAL' THEN amount ELSE 0 END) AS withdrawals
FROM Transactions GROUP BY account_id;

-- Accounts below a balance threshold
SELECT account_id,customer_id,balance FROM Accounts WHERE balance < 1000;
```

UTF-8 SQL MySQL Code example

Expected Output



Note: For the airline task, available seats are calculated from booked tickets. For the hotel task, room availability uses overlapping date ranges, and occupancy is calculated for the selected date.

Task 4 – Airline Reservation Database

Task : Create relational tables with keys/constraints and write the required JOIN, aggregate, and analysis queries.

SQL Code

Task 4 – Airline Reservation Database

SQL > main.sql

```sql
-- MySQL | Task 4: Airline Reservations
CREATE DATABASE AirlineDB;
USE AirlineDB;

CREATE TABLE Flights (
  flight_id INT PRIMARY KEY AUTO_INCREMENT,
  flight_number VARCHAR(20) NOT NULL UNIQUE,
  origin VARCHAR(80) NOT NULL,
  destination VARCHAR(80) NOT NULL,
  total_seats INT NOT NULL CHECK(total_seats > 0)
);
CREATE TABLE Passengers (
  passenger_id INT PRIMARY KEY AUTO_INCREMENT,
  passenger_name VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE
);
CREATE TABLE Bookings (
  booking_id INT PRIMARY KEY AUTO_INCREMENT,
  flight_id INT NOT NULL,
  passenger_id INT NOT NULL,
  booking_date DATE NOT NULL,
  FOREIGN KEY(flight_id) REFERENCES Flights(flight_id),
  FOREIGN KEY(passenger_id) REFERENCES Passengers(passenger_id)
);
CREATE TABLE Tickets (
  ticket_id INT PRIMARY KEY AUTO_INCREMENT,
  booking_id INT NOT NULL UNIQUE,
  seat_number VARCHAR(10) NOT NULL,
  ticket_status ENUM('BOOKED','CANCELLED') NOT NULL DEFAULT 'BOOKED',
  FOREIGN KEY(booking_id) REFERENCES Bookings(booking_id),
  UNIQUE(seat_number, booking_id)
);

INSERT INTO Flights(flight_number,origin,destination,total_seats)
VALUES ('AI101','Hyderabad','Delhi',3),('AI202','Delhi','Mumbai',2);
INSERT INTO Passengers(passenger_name,email) VALUES
('Neha','neha@demo.com'),('Vikram','vikram@demo.com'),('John','john@demo.com');
INSERT INTO Bookings(flight_id,passenger_id,booking_date) VALUES
(1,1,'2026-10-20'),(1,2,'2026-10-21');
INSERT INTO Tickets(booking_id,seat_number,ticket_status) VALUES
(1,'1A','BOOKED'),(2,'1B','BOOKED');

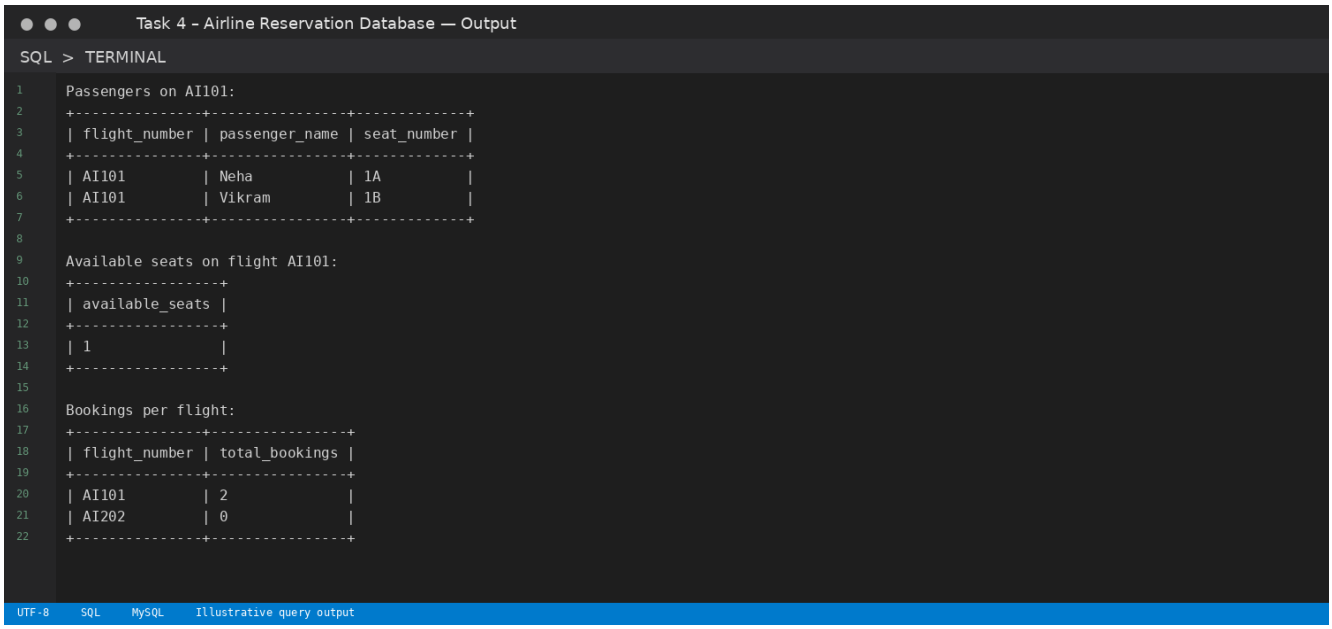
-- Passengers on a flight
SELECT f.flight_number,p.passenger_name,t.seat_number
FROM Flights f JOIN Bookings b ON f.flight_id=b.flight_id
JOIN Passengers p ON b.passenger_id=p.passenger_id
JOIN Tickets t ON b.booking_id=t.booking_id
WHERE f.flight_number='AI101' AND t.ticket_status='BOOKED';

-- Available seats (example flight 1)
SELECT total_seats - (SELECT COUNT(*) FROM Bookings b
 JOIN Tickets t ON b.booking_id=t.booking_id
 WHERE b.flight_id=f.flight_id AND t.ticket_status='BOOKED') AS available_seats
FROM Flights f WHERE f.flight_id=1;

-- Booking count per flight
SELECT f.flight_number,COUNT(b.booking_id) AS total_bookings
FROM Flights f LEFT JOIN Bookings b ON f.flight_id=b.flight_id
GROUP BY f.flight_id,f.flight_number;
```

UTF-8 SQL MySQL Code example

Expected Output



Note: For the airline task, available seats are calculated from booked tickets. For the hotel task, room availability uses overlapping date ranges, and occupancy is calculated for the selected date.

Task 5 – Transport Management Database

Task : Create relational tables with keys/constraints and write the required JOIN, aggregate, and analysis queries.

SQL Code

Task 5 – Transport Management Database

SQL > main.sql

```sql
-- MySQL | Task 5: Transport Management
CREATE DATABASE TransportDB;
USE TransportDB;

CREATE TABLE Vehicles (
  vehicle_id INT PRIMARY KEY AUTO_INCREMENT,
  registration_no VARCHAR(20) NOT NULL UNIQUE,
  vehicle_type VARCHAR(50) NOT NULL
);
CREATE TABLE Routes (
  route_id INT PRIMARY KEY AUTO_INCREMENT,
  route_name VARCHAR(100) NOT NULL UNIQUE,
  distance_km DECIMAL(8,2) NOT NULL CHECK(distance_km > 0)
);
CREATE TABLE Drivers (
  driver_id INT PRIMARY KEY AUTO_INCREMENT,
  driver_name VARCHAR(100) NOT NULL,
  phone VARCHAR(20) NOT NULL UNIQUE
);
CREATE TABLE Assignments (
  assignment_id INT PRIMARY KEY AUTO_INCREMENT,
  vehicle_id INT NOT NULL,
  route_id INT NOT NULL,
  driver_id INT NOT NULL,
  trip_date DATE NOT NULL,
  FOREIGN KEY(vehicle_id) REFERENCES Vehicles(vehicle_id),
  FOREIGN KEY(route_id) REFERENCES Routes(route_id),
  FOREIGN KEY(driver_id) REFERENCES Drivers(driver_id)
);

INSERT INTO Vehicles(registration_no,vehicle_type) VALUES
('TS09AB1234','Bus'),('TS09CD5678','Van');
INSERT INTO Routes(route_name,distance_km) VALUES
('Campus Route',18.5),('Airport Route',32.0);
INSERT INTO Drivers(driver_name,phone) VALUES
('Suresh','9000000001'),('Meena','9000000002');
INSERT INTO Assignments(vehicle_id,route_id,driver_id,trip_date) VALUES
(1,1,1,'2026-10-20'),(1,1,1,'2026-10-21'),(2,2,2,'2026-10-21');

-- Vehicles assigned to a route
SELECT v.registration_no,v.vehicle_type,r.route_name
FROM Assignments a JOIN Vehicles v ON a.vehicle_id=v.vehicle_id
JOIN Routes r ON a.route_id=r.route_id
WHERE r.route_name='Campus Route';

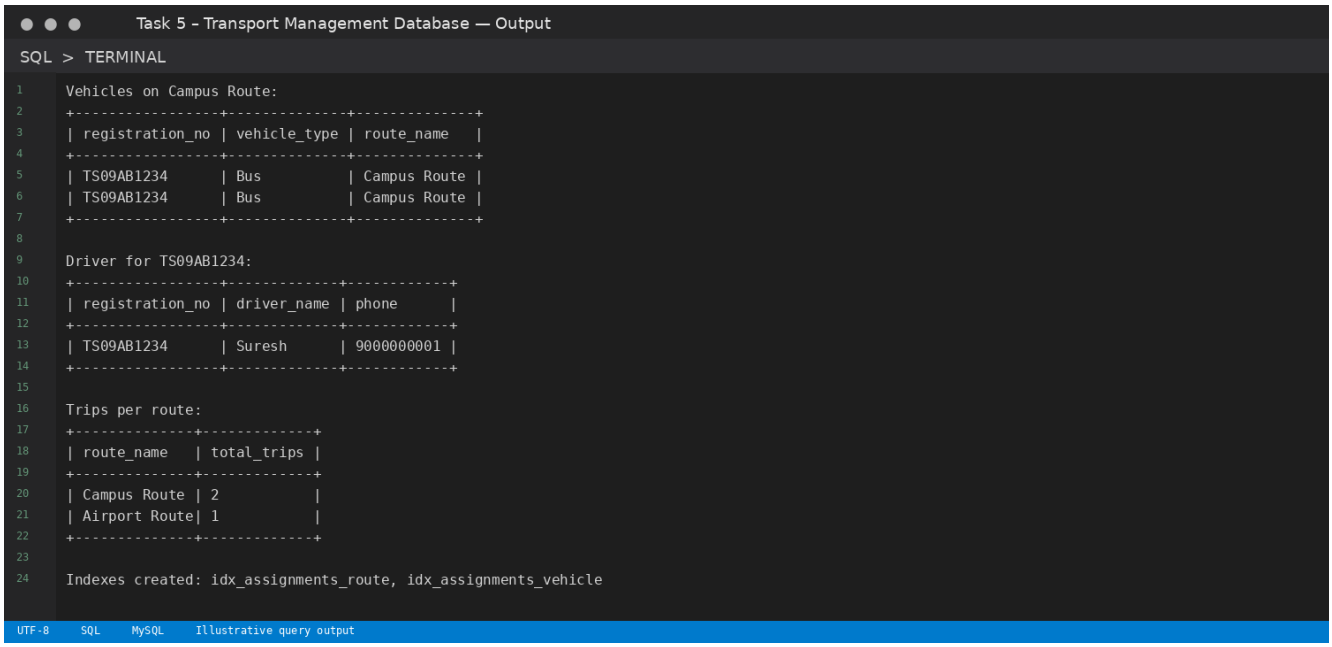
-- Driver details for a vehicle
SELECT v.registration_no,d.driver_name,d.phone
FROM Assignments a JOIN Vehicles v ON a.vehicle_id=v.vehicle_id
JOIN Drivers d ON a.driver_id=d.driver_id
WHERE v.registration_no='TS09AB1234'
ORDER BY a.trip_date DESC LIMIT 1;

-- Trips per route
SELECT r.route_name,COUNT(a.assignment_id) AS total_trips
FROM Routes r LEFT JOIN Assignments a ON r.route_id=a.route_id
GROUP BY r.route_id,r.route_name;

-- Indexes for frequently filtered/joined columns
CREATE INDEX idx_assignments_route ON Assignments(route_id);
CREATE INDEX idx_assignments_vehicle ON Assignments(vehicle_id);
```

UTF-8 SQL MySQL Code example

Expected Output



Note: For the airline task, available seats are calculated from booked tickets. For the hotel task, room availability uses overlapping date ranges, and occupancy is calculated for the selected date.

Task 6 – Hotel Reservation Database

Task : Create relational tables with keys/constraints and write the required JOIN, aggregate, and analysis queries.

SQL Code

Task 6 – Hotel Reservation Database

SQL > main.sql

```sql
-- MySQL | Task 6: Hotel Reservations
CREATE DATABASE HotelDB;
USE HotelDB;

CREATE TABLE Hotels (
  hotel_id INT PRIMARY KEY AUTO_INCREMENT,
  hotel_name VARCHAR(120) NOT NULL,
  city VARCHAR(80) NOT NULL
);
CREATE TABLE Rooms (
  room_id INT PRIMARY KEY AUTO_INCREMENT,
  hotel_id INT NOT NULL,
  room_number VARCHAR(10) NOT NULL,
  room_type VARCHAR(50) NOT NULL,
  nightly_rate DECIMAL(10,2) NOT NULL CHECK(nightly_rate >= 0),
  FOREIGN KEY(hotel_id) REFERENCES Hotels(hotel_id),
  UNIQUE(hotel_id,room_number)
);
CREATE TABLE Guests (
  guest_id INT PRIMARY KEY AUTO_INCREMENT,
  guest_name VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE
);
CREATE TABLE Reservations (
  reservation_id INT PRIMARY KEY AUTO_INCREMENT,
  room_id INT NOT NULL,
  guest_id INT NOT NULL,
  check_in DATE NOT NULL,
  check_out DATE NOT NULL,
  reservation_status ENUM('BOOKED','CANCELLED') NOT NULL DEFAULT 'BOOKED',
  CHECK(check_out > check_in),
  FOREIGN KEY(room_id) REFERENCES Rooms(room_id),
  FOREIGN KEY(guest_id) REFERENCES Guests(guest_id)
);

INSERT INTO Hotels(hotel_name,city) VALUES
('Sunrise Hotel','Hyderabad'),('Lake View','Warangal');
INSERT INTO Rooms(hotel_id,room_number,room_type,nightly_rate) VALUES
(1,'101','Single',2000),(1,'102','Double',3500),(2,'201','Single',1800);
INSERT INTO Guests(guest_name,email) VALUES
('Priya','priya@demo.com'),('Rahul','rahul@demo.com');
INSERT INTO Reservations(room_id,guest_id,check_in,check_out,reservation_status)
VALUES (1,1,'2026-11-01','2026-11-03','BOOKED'),
       (2,2,'2026-11-01','2026-11-02','BOOKED');

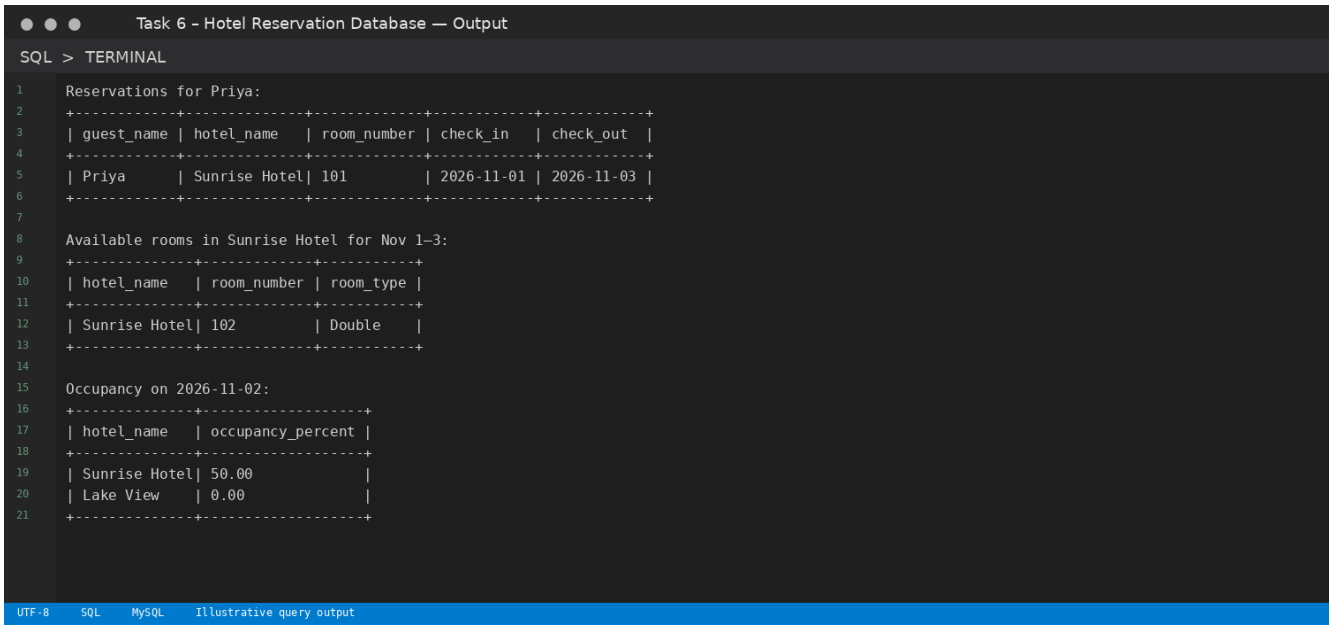
-- Reservations for a guest
SELECT g.guest_name,h.hotel_name,r.room_number,
       res.check_in,res.check_out
FROM Reservations res JOIN Guests g ON res.guest_id=g.guest_id
JOIN Rooms r ON res.room_id=r.room_id
JOIN Hotels h ON r.hotel_id=h.hotel_id
WHERE g.guest_id=1;

-- Rooms available for the entire date range
SELECT h.hotel_name,r.room_number,r.room_type
FROM Rooms r JOIN Hotels h ON r.hotel_id=h.hotel_id
WHERE h.hotel_id=1
AND NOT EXISTS (
 SELECT 1 FROM Reservations res
 WHERE res.room_id=r.room_id AND res.reservation_status='BOOKED'
 AND res.check_in < '2026-11-03'
 AND res.check_out > '2026-11-01'
);

-- Occupancy rate by hotel for a chosen date
SELECT h.hotel_name,
 ROUND(100.0 * COUNT(DISTINCT CASE WHEN res.reservation_status='BOOKED'
 AND res.check_in <= '2026-11-02'
 AND res.check_out > '2026-11-02' THEN r.room_id END)
 / NULLIF(COUNT(DISTINCT r.room_id),0),2) AS occupancy_percent
FROM Hotels h JOIN Rooms r ON h.hotel_id=r.hotel_id
LEFT JOIN Reservations res ON r.room_id=res.room_id
GROUP BY h.hotel_id,h.hotel_name;
```

UTF-8 SQL MySQL Code example

Expected Output



Note: For the airline task, available seats are calculated from booked tickets. For the hotel task, room availability uses overlapping date ranges, and occupancy is calculated for the selected date.